

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Mechanical Engineering

PHYSICS-INFORMED NEURAL NETWORK FOR INTERNAL PIPE FLOW SIMULATION

A computational study of steady, incompressible flow in a cylindrical pipe

Prepared by

Mohammadreza Sotoudeh

Course: Fluid Mechanics I

Instructor: Dr. Karimi

Spring 2025

Abstract

This project investigates the use of a physics-informed neural network (PINN) to represent flow inside a cylindrical pipe. The model uses cylindrical coordinates and returns the radial, azimuthal, and axial velocity components together with pressure. A neural network is trained in PyTorch using collocation points sampled in the pipe domain, with a continuity-residual term and inlet/outlet boundary-condition terms included in the loss function. The report examines contour plots of the predicted velocity components and pressure, the pressure distribution along the pipe centreline, and a friction-factor estimate derived from the predicted pressure gradient. The reported training loss generally decreases over 10,000 Adam optimization epochs, although several intermittent increases occur. The source implementation should be interpreted as a preliminary physics-informed model rather than a complete Navier–Stokes PINN: its loss function does not include the momentum-equation residuals or an explicit no-slip wall condition, despite those elements being discussed in the Persian report.

Contents

1. Introduction
 2. Problem Definition and Assumptions
 3. Governing Equations and Model Formulation
 4. Neural-Network Architecture and Training
 5. Predicted Velocity Field
 6. Pressure Field and Centreline Pressure
 7. Friction-Factor Estimate
 8. Training History
 9. Discussion and Limitations
 10. Conclusions
- References
- Appendix A. Python Implementation

1. Introduction

Numerical modeling of fluid flow is a central task in mechanical engineering and is used in the design and optimization of piping systems, transport networks, and many other engineering applications. Conventional computational methods—including the finite-difference, finite-volume, and finite-element methods—typically rely on a discretized computational domain. The quality of the mesh and the cost of solving the resulting system can become important considerations, particularly for complicated geometries or demanding boundary conditions.

Physics-informed neural networks (PINNs) are a machine-learning approach in which physical laws, commonly expressed as partial differential equations (PDEs), are incorporated into the training objective. A neural network is trained not only to fit available data but also to reduce the residuals of the governing equations and satisfy prescribed conditions.

The project applies this idea to a steady, incompressible flow problem in a straight cylindrical pipe. The network takes the cylindrical coordinates (r , θ , z) as inputs and predicts the three velocity components and pressure. The original report presents contour plots and a centreline pressure profile, and uses the pressure gradient to estimate a Darcy-type friction factor.

2. Problem Definition and Assumptions

The computational domain is a straight cylindrical pipe of radius $R = 0.01$ m and length $L = 1.0$ m. The flow is treated as steady, incompressible, and laminar. The fluid density and dynamic viscosity are set to 1 kg/m^3 and $1 \times 10^{-3} \text{ Pa}\cdot\text{s}$, respectively, and the prescribed inlet axial velocity is 1 m/s .

2.1 Physical assumptions

- Steady flow: the modeled quantities do not explicitly vary with time.
- Incompressible fluid: density is treated as constant.
- Newtonian fluid: dynamic viscosity is assumed constant.
- Homogeneous domain: no change in pipe geometry or fluid properties along the pipe is modeled.
- Cylindrical geometry: the pipe axis is aligned with the z direction.

2.2 Boundary conditions described in the source report

- Inlet ($z = 0$): the axial velocity is prescribed as $u_z = u_0 = 1 \text{ m/s}$.
- Outlet ($z = L$): the pressure is set to $p = 0$ (reference pressure).
- Pipe wall ($r = R$): the report describes a no-slip condition. However, this condition is not explicitly included in the provided code's loss function.

2.3 Model parameters

Parameter	Symbol	Value	Unit / note
Pipe radius	R	0.01	m
Pipe length	L	1.0	m
Pipe diameter	$D = 2R$	0.02	m
Dynamic viscosity	μ	1×10^{-3}	$\text{Pa}\cdot\text{s}$
Density	ρ	1.0	kg/m^3
Inlet axial velocity	u_0	1.0	m/s
Interior collocation points	N_c	10,000	Latin-hypercube sampling
Training epochs	N_{epochs}	10,000	Adam optimizer
Learning rate	η	1×10^{-3}	Adam

2.4 Coordinate system and outputs

The model uses cylindrical coordinates. The input vector is $\mathbf{x} = [r, \theta, z]$ and the four network outputs are $[u_r, u_\theta, u_z, p]$, representing radial velocity, azimuthal velocity, axial velocity, and pressure. The report describes the domain as 3D, with $0 \leq r \leq R$, $0 \leq \theta \leq 2\pi$, and $0 \leq z \leq L$.

Variable	Description	Range
r	Radial coordinate	$0 \leq r \leq 0.01 \text{ m}$
θ	Azimuthal angle	$0 \leq \theta \leq 2\pi \text{ rad}$
z	Axial coordinate	$0 \leq z \leq 1.0 \text{ m}$
u_r	Radial velocity component	Predicted by network
u_θ	Azimuthal velocity component	Predicted by network
u_z	Axial velocity component	Predicted by network
p	Pressure	Predicted by network

3. Governing Equations and Model Formulation

For incompressible flow, mass conservation requires the velocity field to be divergence-free. In cylindrical coordinates, the continuity equation is

$$\partial u_r / \partial r + u_r / r + (1/r) \partial u_\theta / \partial \theta + \partial u_z / \partial z = 0$$

The source code uses automatic differentiation in PyTorch to calculate derivatives of the predicted velocity components with respect to the input coordinates. The continuity residual is then evaluated at collocation points.

The Persian report discusses the Navier–Stokes momentum equations and the continuity equation as the governing physics. In the code actually supplied, however, the loss function only includes the continuity residual, the inlet axial-velocity condition, and the outlet pressure condition. The radial and axial momentum residuals, the azimuthal momentum residual, and the no-slip wall loss are not implemented. Accordingly, the implementation shown in the appendix is not a complete Navier–Stokes PINN.

4. Neural-Network Architecture and Training

The provided Python class defines a fully connected feed-forward neural network. Its architecture, as written in the code, contains an input layer with three features, two hidden layers with 64 neurons each and tanh activations, and an output layer with four values.

Layer	Input / output size	Activation
Input	3: r, θ, z	—
Hidden layer 1	64	tanh
Hidden layer 2	64	tanh
Output	4: u_r, u_θ, u_z, p	Linear

The Persian report mentions four hidden layers, but the code contains only two hidden Linear layers of width 64. This report follows the actual implementation when describing the architecture.

4.1 Collocation points

The code generates $N_c = 10,000$ interior points using Latin-hypercube sampling through the pyDOE package. The radial coordinate is sampled with a square-root transformation, $r = R\sqrt{\xi}$, while θ and z are sampled over their respective ranges.

4.2 Loss function

The implemented objective can be summarized as

$$L = \text{mean}(\text{continuity residual}^2) + L_{\text{inlet}} + L_{\text{outlet}}$$

The inlet term penalizes deviation of u_z from u_0 at $z = 0$. The outlet term penalizes nonzero predicted pressure at $z = L$. The optimizer is Adam with a learning rate of 10^{-3} , and the training loop runs for 10,000 epochs. The loss value is printed every 500 epochs.

5. Predicted Velocity Field

The source report plots the predicted velocity components on a grid spanning the radial and axial directions at a fixed azimuthal angle $\theta = \pi/2$. These visualizations are useful for inspecting the model output, but they should not be treated as validated physical solutions without the missing momentum equations and wall condition.

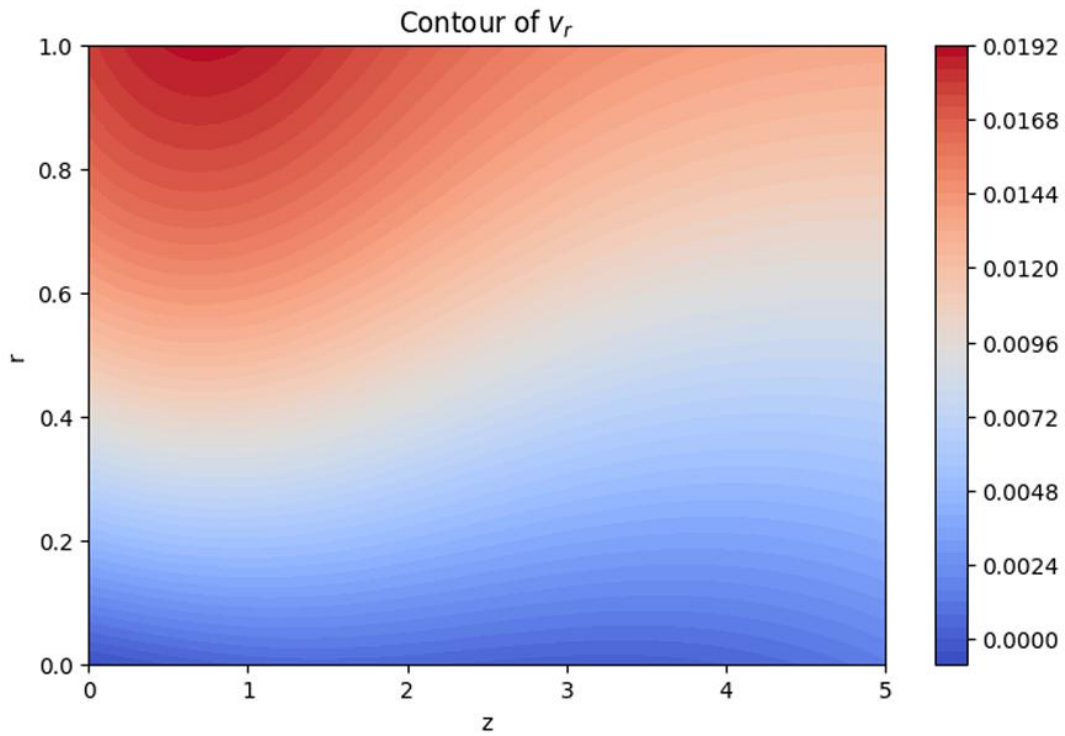


Figure 1. Radial velocity component, u_r , reproduced from the original report.

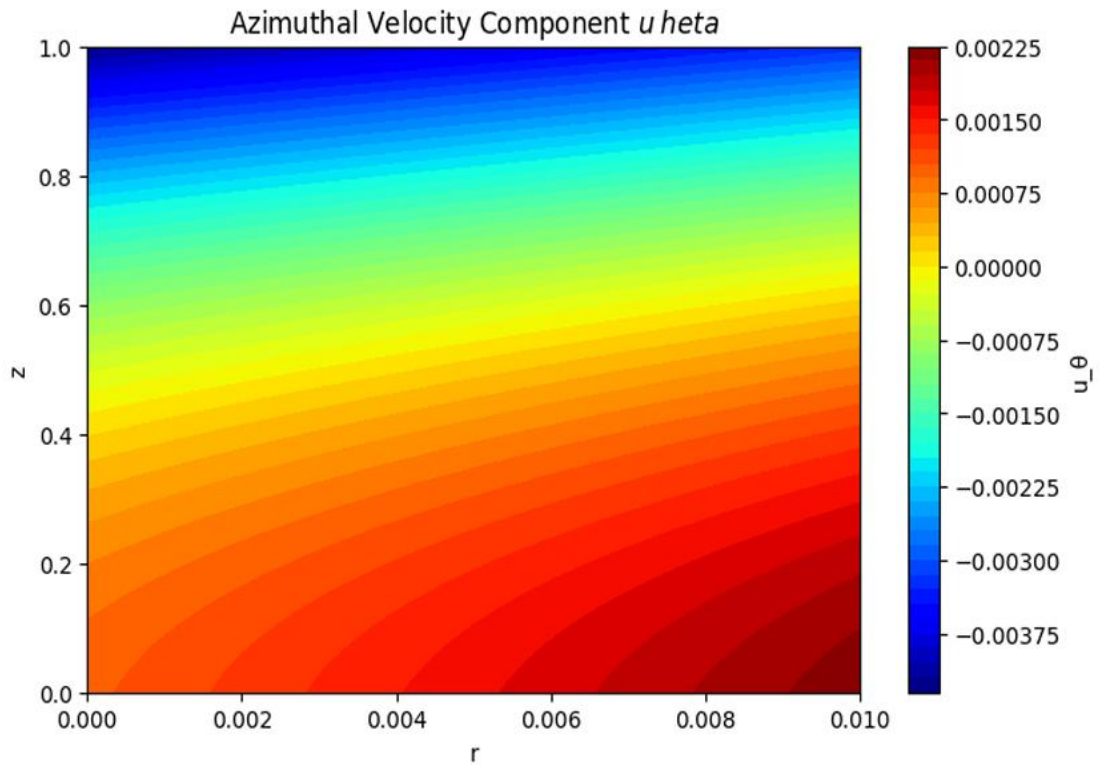


Figure 2. Azimuthal velocity component, u_{θ} , reproduced from the original report.

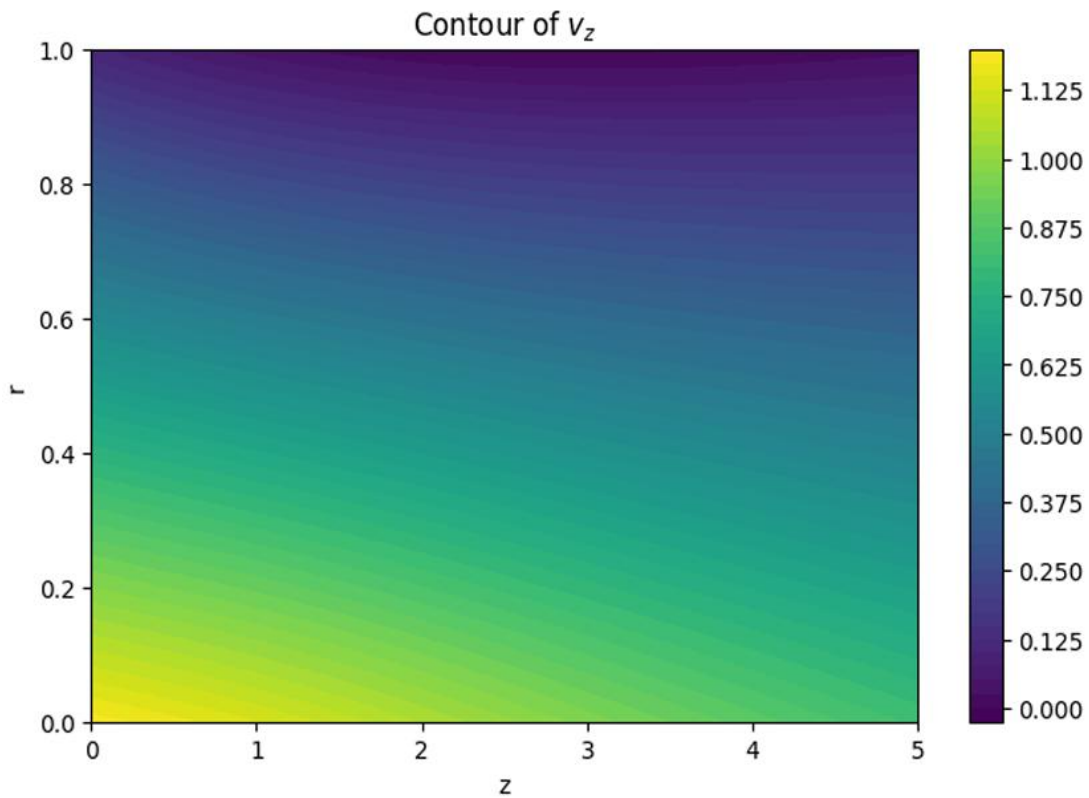


Figure 3. Axial velocity component, u_z , reproduced from the original report.

5.1 Radial velocity

The original report interprets the radial-velocity contour as having relatively small magnitudes compared with the axial component, and notes variation across the radius and along the pipe. The plotted range is approximately 0 to 0.0192 in the displayed color scale. Because the wall no-slip condition is not included in the code, the plot alone cannot establish that the wall condition is satisfied.

5.2 Azimuthal velocity

For ideal, fully developed flow in a straight pipe without swirl, the azimuthal velocity would be expected to be negligible. The source report notes that a nonuniform azimuthal-velocity contour may indicate deviation from that ideal behavior. The current loss function does not explicitly constrain u_θ through a momentum equation or a zero-swirl condition.

5.3 Axial velocity

The source report discusses a pipe-flow-like axial velocity profile, with higher values near the centre and lower values near the wall, and reports values up to approximately 1.125 in the displayed field. It also compares this qualitative pattern with the classical Hagen–Poiseuille profile. However, the supplied implementation does not impose the wall condition or the axial momentum equation, so the resemblance should be considered qualitative rather than a validated recovery of fully developed pipe flow.

6. Pressure Field and Centreline Pressure

Pressure is the fourth output of the network. The outlet condition sets the predicted pressure to zero at $z = L$. The original report presents a pressure contour and a pressure profile sampled along the pipe centreline ($r = 0$, $\theta = \pi/2$).

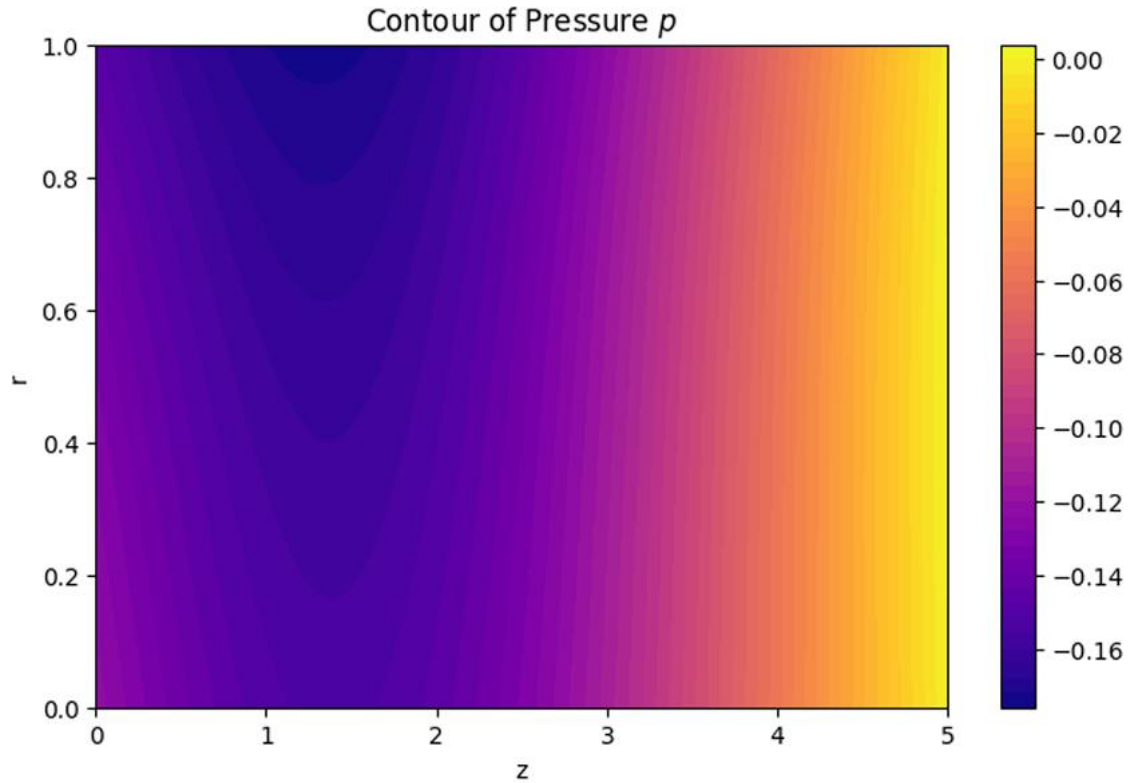


Figure 4. Predicted pressure contour, reproduced from the original report.

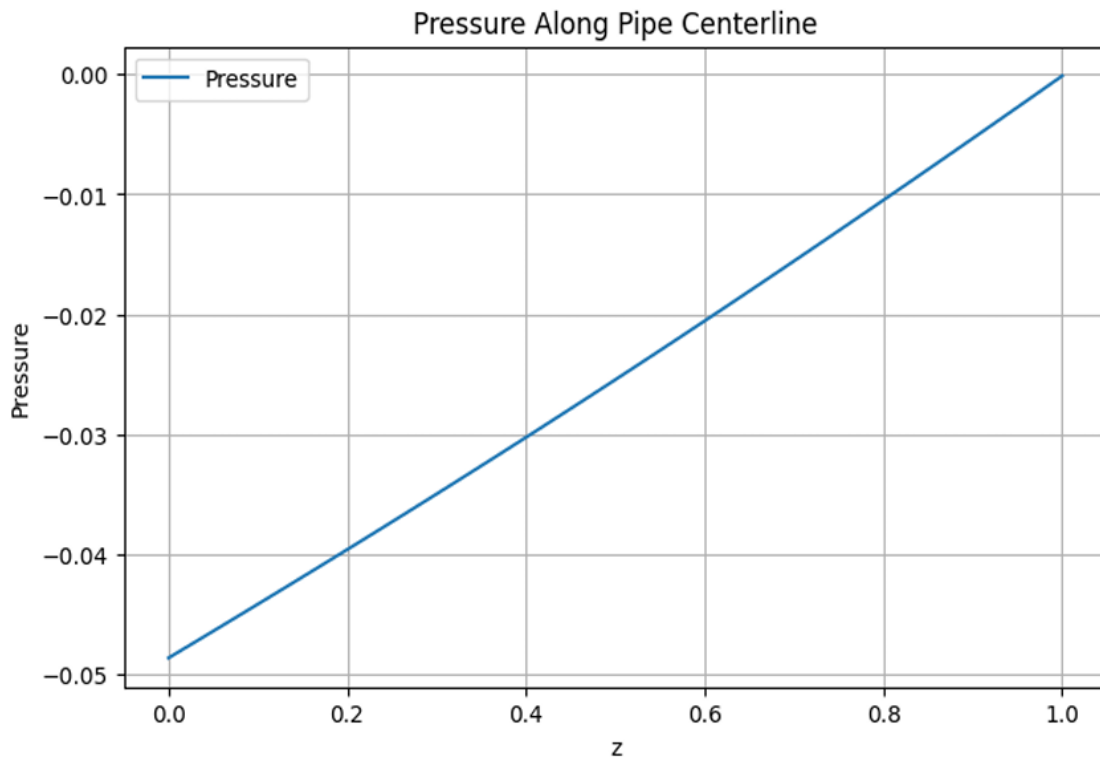


Figure 5. Pressure distribution along the pipe centreline, reproduced from the original report.

6.1 Interpretation

The report interprets the pressure as decreasing from inlet to outlet and as varying comparatively little with radius. Such a trend is qualitatively consistent with pressure-driven flow in a straight pipe. Nevertheless, the source implementation does not include momentum-equation residuals linking pressure gradients to viscous and inertial forces. Therefore, the pressure field is not guaranteed to satisfy the Navier–Stokes equations, and no quantitative agreement with an analytical or benchmark solution is established.

7. Friction-Factor Estimate

The code estimates a pressure gradient along the centreline using NumPy's numerical gradient function. It then evaluates a Darcy-type friction-factor expression at each axial location:

$$f = D \left| \frac{dp}{dz} \right| / [0.5 \rho (\max(u_z))^2]$$

Here D is the pipe diameter, ρ is the density, dp/dz is the estimated centreline pressure gradient, and $\max(u_z)$ is taken from the predicted axial-velocity grid. This is the formula used in the source code. Since the predicted pressure field is not constrained by the momentum equations, the resulting friction factor should be treated as a preliminary estimate, not as a validated friction-factor prediction.

8. Training History

The original report records the loss every 500 epochs. The reported values are reproduced below. Although the overall trend is downward, the loss shows several pronounced temporary increases, including at epochs 5,000, 8,000, and 9,000. The final printed value is 0.001307 at epoch 9,500.

Epoch	Loss
0	1559.238525
500	0.036582
1000	0.020284
1500	0.014338
2000	0.038168
2500	0.008535
3000	0.006611
3500	0.005963
4000	0.004592
4500	0.003689
5000	0.104235
5500	0.003141
6000	0.002479
6500	0.001997
7000	0.002034
7500	0.001550
8000	0.011949
8500	0.001441
9000	0.474289
9500	0.001307

9. Discussion and Limitations

The project demonstrates a basic PINN workflow: defining a neural network, sampling collocation points, using automatic differentiation, applying boundary-condition penalties, training with Adam, and visualizing the predicted fields. These are useful components for developing physics-informed computational models.

Several limitations in the source code are important for interpretation:

- The loss function includes the continuity equation but does not include the Navier–Stokes momentum residuals.
- The no-slip condition at the pipe wall is discussed in the report but is not enforced in the supplied loss function.
- The code samples the angular coordinate, but the displayed contour plots are evaluated at a single fixed angle, $\theta = \pi/2$.
- The pressure-gradient/friction-factor calculation is not validated against an analytical pipe-flow solution or a trusted CFD result.
- The report's stated four-hidden-layer architecture differs from the two hidden layers visible in the supplied code.
- The source code contains formatting/indentation inconsistencies in the PDF extraction; the appendix presents a cleaned, readable transcription of the same implementation logic.

Recommended next steps are to add the cylindrical-coordinate momentum residuals, explicitly enforce no-slip at $r = R$, handle the coordinate singularity at $r = 0$ carefully, and compare the resulting velocity and pressure fields against the Hagen–Poiseuille analytical solution or a verified CFD benchmark.

10. Conclusions

A feed-forward neural network was used to model velocity and pressure fields for flow in a cylindrical pipe. The provided implementation combines a continuity residual with inlet and outlet boundary-condition penalties and is trained for 10,000 epochs using Adam. The source report includes contour plots for the velocity components and pressure, a centreline pressure plot, and a friction-factor estimate based on the predicted pressure gradient.

The results illustrate the workflow and visualization capabilities of a PINN, but the current code does not yet constitute a complete physics-informed solution of the Navier–Stokes equations. Implementing the momentum equations and wall boundary condition, followed by quantitative validation, is necessary before drawing firm conclusions about predictive accuracy.

References

- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.
- Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3, 422–440.
- White, F. M. (2006). *Viscous Fluid Flow* (3rd ed.). McGraw-Hill.
- Kundu, P. K., Cohen, I. M., & Dowling, D. R. (2016). *Fluid Mechanics* (6th ed.). Academic Press.
- Wang, S., Yu, X., & Perdikaris, P. (2021). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449, 110768.

Appendix A. Python Implementation

The listing below is a cleaned, English-commented transcription of the implementation included in the source PDF. The objective and boundary conditions are preserved; the missing momentum residuals and wall no-slip term are not added.

```
# Install dependency in the environment if needed:
# pip install pyDOE

import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from pyDOE import lhs

# Domain and physical parameters
R = 0.01      # Pipe radius (m)
L = 1.0       # Pipe length (m)
mu = 1e-3     # Dynamic viscosity (Pa.s); defined in source, not used in loss
rho = 1.0     # Density (kg/m^3)
D = 2 * R     # Pipe diameter (m)
u0 = 1.0      # Prescribed inlet axial velocity (m/s)

# PINN for velocity components and pressure
class PINN(nn.Module):
    def __init__(self):
        super(PINN, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(3, 64),
            nn.Tanh(),
            nn.Linear(64, 64),
            nn.Tanh(),
            nn.Linear(64, 4) # Outputs: u_r, u_theta, u_z, p
        )

    def forward(self, x):
        return self.net(x)

# Number of interior collocation points
N_c = 10000

# Sample collocation points in cylindrical coordinates
r_c = R * np.sqrt(lhs(1, N_c))
theta_c = 2 * np.pi * lhs(1, N_c)
z_c = L * lhs(1, N_c)
X_c = np.hstack((r_c, theta_c, z_c))
X_c_tensor = torch.tensor(X_c, dtype=torch.float32)

# Inlet boundary condition: u_z = u0 at z = 0
def inlet_condition(model, N=1000):
    r_vals = torch.linspace(0, R, N).view(-1, 1)
    z_in = torch.zeros_like(r_vals)
    theta_in = torch.full_like(r_vals, np.pi / 2)
    X_inlet = torch.cat([r_vals, theta_in, z_in], dim=1)
    u_z_pred = model(X_inlet)[0, 2]
    u_z_target = torch.full_like(u_z_pred, u0)
    return torch.mean((u_z_pred - u_z_target) ** 2)
```

```

# Outlet boundary condition:  $p = 0$  at  $z = L$ 
def outlet_condition(model, N=1000):
    r_vals = torch.linspace(0, R, N).view(-1, 1)
    z_out = torch.full_like(r_vals, L)
    theta_out = torch.full_like(r_vals, np.pi / 2)
    X_outlet = torch.cat([r_vals, theta_out, z_out], dim=1)
    p_pred = model(X_outlet)[:, 3]
    return torch.mean(p_pred ** 2)

# Loss: continuity residual plus inlet and outlet penalties
def loss_function(model, X_c):
    X_c.requires_grad_(True)
    output = model(X_c)
    u_r = output[:, 0]
    u_theta = output[:, 1]
    u_z = output[:, 2]
    p = output[:, 3]

    r = X_c[:, 0]
    theta = X_c[:, 1]
    z = X_c[:, 2]

    # Derivatives of radial velocity
    grads = torch.autograd.grad(
        u_r, X_c, torch.ones_like(u_r), create_graph=True
    )[0]
    du_r_dr = grads[:, 0]
    du_r_dtheta = grads[:, 1] / r
    du_r_dz = grads[:, 2]

    # Derivatives of azimuthal velocity
    grads = torch.autograd.grad(
        u_theta, X_c, torch.ones_like(u_theta), create_graph=True
    )[0]
    du_theta_dr = grads[:, 0]
    du_theta_dtheta = grads[:, 1] / r
    du_theta_dz = grads[:, 2]

    # Derivatives of axial velocity
    grads = torch.autograd.grad(
        u_z, X_c, torch.ones_like(u_z), create_graph=True
    )[0]
    du_z_dr = grads[:, 0]
    du_z_dtheta = grads[:, 1] / r
    du_z_dz = grads[:, 2]

    # Derivatives of pressure (computed in the source code)
    grads = torch.autograd.grad(
        p, X_c, torch.ones_like(p), create_graph=True
    )[0]
    dp_dr = grads[:, 0]
    dp_dtheta = grads[:, 1] / r
    dp_dz = grads[:, 2]

    # Incompressible continuity equation in cylindrical coordinates
    continuity = du_r_dr + u_r / r + du_theta_dtheta + du_z_dz

    # The source implementation does not include momentum residuals
    # or an explicit no-slip wall condition.
    loss = (
        torch.mean(continuity ** 2)
        + inlet_condition(model)
        + outlet_condition(model)
    )
    return loss

```

```

# Train the network
model = PINN()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10000):
    optimizer.zero_grad()
    loss = loss_function(model, X_c_tensor)
    loss.backward()
    optimizer.step()

    if epoch % 500 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.6f}")

# Evaluate the network on a radial-axial grid at theta = pi/2
r_plot = np.linspace(0, R, 100)
z_plot = np.linspace(0, L, 100)
R_grid, Z_grid = np.meshgrid(r_plot, z_plot, indexing="ij")
Theta_fixed = np.full_like(R_grid, np.pi / 2)

X_plot = np.stack(
    [R_grid.flatten(), Theta_fixed.flatten(), Z_grid.flatten()],
    axis=1
)
X_plot_tensor = torch.tensor(X_plot, dtype=torch.float32)

with torch.no_grad():
    output = model(X_plot_tensor)
    u_r_pred = output[:, 0].numpy().reshape(R_grid.shape)
    u_theta_pred = output[:, 1].numpy().reshape(R_grid.shape)
    u_z_pred = output[:, 2].numpy().reshape(R_grid.shape)
    p_pred = output[:, 3].numpy().reshape(R_grid.shape)

# Contour: radial velocity
plt.figure(figsize=(8, 5))
plt.contourf(R_grid, Z_grid, u_r_pred, levels=50, cmap="jet")
plt.colorbar(label="u_r")
plt.xlabel("r")
plt.ylabel("z")
plt.title("Radial Velocity Component $u_r$")
plt.show()

# Contour: azimuthal velocity
plt.figure(figsize=(8, 5))
plt.contourf(R_grid, Z_grid, u_theta_pred, levels=50, cmap="jet")
plt.colorbar(label="u_theta")
plt.xlabel("r")
plt.ylabel("z")
plt.title("Azimuthal Velocity Component $u_{\theta}$")
plt.show()

# Contour: axial velocity
plt.figure(figsize=(8, 5))
plt.contourf(R_grid, Z_grid, u_z_pred, levels=50, cmap="jet")
plt.colorbar(label="u_z")
plt.xlabel("r")
plt.ylabel("z")
plt.title("Axial Velocity Component $u_z$")
plt.show()

```

```

# Contour: pressure
plt.figure(figsize=(8, 5))
plt.contourf(R_grid, Z_grid, p_pred, levels=50, cmap="viridis")
plt.colorbar(label="p")
plt.xlabel("r")
plt.ylabel("z")
plt.title("Pressure Contour $p$")
plt.show()

# Pressure along the pipe centreline (r = 0)
z_center = np.linspace(0, L, 100)
r_center = np.zeros_like(z_center)
theta_center = np.full_like(z_center, np.pi / 2)
X_centerline = np.stack(
    [r_center, theta_center, z_center], axis=1
)
X_centerline_tensor = torch.tensor(X_centerline, dtype=torch.float32)

with torch.no_grad():
    p_centerline = model(X_centerline_tensor)[: , 3].numpy()

plt.figure(figsize=(8, 5))
plt.plot(z_center, p_centerline, label="Pressure")
plt.xlabel("z")
plt.ylabel("Pressure")
plt.title("Pressure Along Pipe Centerline")
plt.grid(True)
plt.legend()
plt.show()

# Friction-factor estimate from the pressure gradient
pressure_gradient = np.gradient(p_centerline, z_center)
friction_factor = (
    D * np.abs(pressure_gradient)
    / (0.5 * rho * np.max(u_z_pred) ** 2)
)

plt.figure(figsize=(8, 5))
plt.plot(z_center, friction_factor, label="Friction Factor")
plt.xlabel("z")
plt.ylabel("f")
plt.title("Friction Factor Along Pipe")
plt.grid(True)
plt.legend()
plt.show()

```